Module 16 - Assignment

S M Wasir Jayed Rafi
ID: 01410369963
Full Stack Web Development with Python, Django, React & AI
Submission Date: 21/07/26

1. What is Django? Briefly explain main purpose

Ans: Django is a free, open-source web framework written in Python that's used to build websites and web apps quickly. It was released in 2005 and was originally built to help manage news websites, which is why it can handle content-heavy sites easily. Its main purpose is to make backend development faster by giving a lot of built-in tools like database handling, user login systems, and an admin panel, so you don't have to build everything from scratch. It follows the "don't repeat yourself" idea, meaning less code and fewer bugs.

2. Write 8 features and 5 advantages of Django

Ans:

Features:

1. **Built-in Admin Panel** — automatically generates a dashboard to manage your database.
2. **ORM (Object-Relational Mapping)** — lets you interact with the database using Python instead of writing raw SQL.
3. **MVT Architecture** — separates the app into Model, View and Template for cleaner code.
4. **Built-in Authentication System** — handles login, logout, and user permissions out of the box.
5. **URL Routing** — lets you map URLs to views easily using urls.py.
6. **Built-in Security** — protects against common attacks by default without needing extra setup.
7. **Scalability** — can handle small projects as well as large, high-traffic websites.
8. **Template Engine** — lets you build dynamic HTML pages using Django's own templating language.

Advantages:

1. Saves development time since a lot of features come built-in.
2. Strong security by default, so you don't have to configure everything manually.
3. Good for both small and large projects since it scales well.
4. Has a large community and good documentation, so it's easy to find help.
5. Works well with Python, so it's beginner friendly if you already know Python basics.

3. Explain the Django installation process and the purpose of a virtual environment

Ans: To install Django, you first need Python installed on your system. Then you create a virtual environment, activate it, and install Django using pip inside that environment.

```
python -m venv env                # create virtual environment

env\Scripts\activate              # activate on Windows

source env/bin/activate           # activate on Mac/Linux

pip install django                # install Django

django-admin startproject myproject   # create a project

python manage.py runserver  # run the development server
```

A virtual environment is basically an isolated space for a project's dependencies, separate from your system's global Python packages. It's used so different projects can have different versions of Django or other libraries without conflicting with each other. This keeps things clean and avoids version issues when working on multiple projects.

4. Explain the purpose of the following files: manage.py, settings.py, urls.py, wsgi.py, asgi.py

Ans: `manage.py` is the command-line tool used to run commands for the project, like starting the server, creating apps, or running migrations.

`settings.py` holds all the configuration for the project, like database settings, the list of installed apps, and static file paths.

`urls.py` is where you define the URL patterns for the project, connecting each URL to the view that should handle it.

`wsgi.py` stands for Web Server Gateway Interface, and it's used to run the project on a normal web server when it's deployed.

`asgi.py` stands for Asynchronous Server Gateway Interface, and it's used when the project needs to support newer async features during deployment instead of the traditional way.

5. What is the difference between a Django Project and a Django App? Write at least 5 differences

Ans:

Point	Django Project	Django App
Meaning	The whole website or application as a whole	A smaller module inside the project that handles one specific feature
Command	Created using `django-admin startproject`	Created using `python manage.py startapp`
Contains	Project-level files like settings.py and urls.py	Its own models, views, and templates for that feature
Structure	Can contain multiple apps inside it	Meant to do one specific thing
Reusability	Not really reused the same way across other projects	Can be reused in other projects

6. Briefly explain the following: Models, Views, Templates, URLs, Django Admin, Static Files, Media Files

Ans: Models define the structure of your database, basically representing tables as Python classes. Each field in the model becomes a column in the database.

Views contain the logic that decides what data to send back to the user, like fetching something from the database and returning it as a response.

Templates are the HTML files that define how the page looks, and they can display dynamic data passed in from the views using Django's template syntax.

URLs connect a specific web address to the view that should handle the request, defined inside urls.py, and this connection between URL and view is called routing.

Django Admin is a built-in dashboard that lets you manage your database (add, edit, delete records) without writing extra code, just by registering your models in admin.py.

Static Files are files like CSS, JavaScript, and images that don't change and are used to style and add behavior to the site.

Media Files are files uploaded by users while the app is running, like profile pictures or documents, and they're stored separately from static files.

7. What is the difference between MVC and MVT architecture? Write at least 5 differences

Ans:

Point	**MVC (Model-View-Controller)**	**MVT (Model-View-Template)**
Used in	Frameworks like Laravel or ASP.NET.	Used specifically in Django.
Logic handling	Controller handles the logic and decides what to show.	View itself handles that logic.
UI part	View is only responsible for the UI.	Template is responsible for the UI instead.
Connection	Developer usually connects Model, View and Controller manually.	Django connects everything automatically through URLs.
Routing	Generally needs more manual routing setup.	Django's URL dispatcher handles routing on its own.